Cloud Segmentation Project - Jan's internship Report

Jan Moran Ricardo
Advisor: Dani Ponsa Musarra
Computer Vision Center & Universitat Autonoma de Barcelona
janmoranr@duck.com

Contents

1 Abstract **3**

2 Introduction **3**

3 Dataset **3**
3.1 Image Patches and Regions of Interest . . . 4
3.2 Anotation strategy . . . 4
3.3 Files and Directories . . . 5
3.4 Downloading the Dataset . . . 6
3.4.1 Downloading the Premade Memmaps . . . 6
3.4.2 Downloading batches of ROIs directly. . . . 7
3.4.3 Making our oun memmaps . . . 7

4 Model **8**
4.1 Model Architecture (U-Net) . . . 8
4.1.1 Input Layer . . . 8
4.1.2 Contracting path (Encoder) . . . 9
4.1.3 Bottleneck . . . 9
4.1.4 Expanding path (Decoder) . . . 10
4.1.5 Output Layer . . . 10
4.2 Comparing Different Models . . . 11
4.3 Our Trained Models . . . 12

5 Results **14**
5.1 Quantitative Analysis . . . 14
5.1.1 Scenario 1 . . . 14
5.1.2 Scenario 2 . . . 15
5.1.3 Confusion Matrices . . . 15
5.2 Empirical Analysis . . . 16
5.2.1 Wrong Behaviors . . . 16
5.2.2 Good Behaviors . . . 17
5.3 Upgrades and Changes . . . 19
5.3.1 Training Changes . . . 19
5.3.2 Model Changes . . . 19

6 Repository **19**
6.1 File structure . . . 19
6.2 Code . . . 20
6.2.1 Executable code . . . 20
6.2.2 Changes to the model . . . 20
6.2.3 Loss Functions and Metrics . . . 20
6.2.4 Dataset and Dataloader . . . 20
6.3 Errors . . . 21

1 Abstract

When working with very high-altitude images, clouds often pose significant challenges. Occasionally, an image may contain a cloud, leading us to discard it entirely. Alternatively, if the cloud is relatively small, we might keep the image as it is, hoping it won't alter the future results.

Neither of these approaches is ideal. Discarding a valid image due to a cloud covering half of it results in the loss of valuable data. While this might be acceptable if it happens just once, it becomes more problematic when the number of discards start to be substantial. This situation often leads to the second issue: making compromises by retaining images with a small cloud coverage to minimize data loss. Again this is not great, as it leaves corrupted images on our dataset.

Therefore, this project will undertake the task of making a cloud segmentation model maker to assist all of those projects which have complications regarding "cloudy" images. with projects facing challenges related to "cloudy" images.

2 Introduction

As previously stated, the aim of this project is to create a model capable of segmenting different types of clouds.

Before delving into the specifics of the project, it is important to note a few key points: Firstly, this document should be read as a report, not as a formal research paper. Although the project can be considered complete, there remains significant room for improvement. This is primarily because the project was developed by a student during a five-month internship.

It is also important to understand that the student, who is the author of this report, as well as the code and other documentation, is taking their first steps into the world of research and AI. This project can even be considered his first IA related one. This should not be viewed as an excuse but rather as a remainder that the project may have limitations in certain aspects. If you are reading this as an example of how AI should be coded, please be aware of this context. Nonetheless, the project is functional and may prove useful to many people.

Back to the report: in it we will present all aspects of the project's development. We will not only discuss what has been accomplished but also explore potential future directions for the project. This document includes:

- Information about the dataset, focusing on what's relevant for the project and how to use it.
- Details about the model, the metrics, and the loss functions implemented.
- An explanation of the repository's structure, including the different functions, scripts, file organization, and potential errors we have deemed necessary to address.
- An analysis of our model's results and a comparison with other models.
- A list of potential improvements that could be made to the project but have not yet been implemented.

Having said all this, we can now begin.

3 Dataset

The dataset we've worked with is the CloudSen12 dataset. CloudSen12 is a new multi-temporal global dataset to foster research in cloud and cloud shadow detection, it has 49,400 image patches, including:

- Sentinel-2 level-1C and level-2A multi-spectral data.
- Sentinel-1 synthetic aperture radar data.

- Auxiliary remote sensing products.
- Different hand-crafted annotations to label the presence of thick and thin clouds and cloud shadows
- The results from eight state-of-the-art cloud detection algorithms (CDAs).

In this document, I will briefly explain the dataset, focusing only on aspects relevant to this project. The main reason beeing that this is very well documented dataset on it's own. If you are interested in more specific details, I strongly recommend reading the associated paper[1] and exploring how the Sentinel 1 and 2 bands can be combined[2].

3.1 Image Patches and Regions of Interest

An image patch (IP) comprises a singular multispectral image depicting a specific geographical area on Earth. This area is referred to as a Region of Interest (ROI), with each ROI being linked to five IPs. This means that we will dispose of multiple images of the same place but temporarily displaced, meaning that they will have the same background but be covered with different types of clouds and shadows.

Originally there were 20,000 ROIs but after a very harsh selection, the creators of the dataset have reduced that number to 9,880 ROIs. This selection in turn guarantees us that for each ROI there will be at least 1 IP with one of the following cloud coverages: cloud-free (0%), almost-clear (0–25%), low-cloudy (25–65%), mid-cloudy (45–65%), and cloudy image (>65%).

Back to the IPs, they have a size of 509x509 pixels and a resolution ranging from 10 to 60 meters, depending on the band, there beeing a total of 13 bands. Each image covers therefore an area approximately equivalent to 5090x5090 meters.

3.2 Anotation strategy

After selecting the ROIs and IPs, the CloudSen12 team, comprised of 14 distinct cloud detection experts (CDE), commenced the manual annotation phase. The strategy employed to annotate the different IPs can be divided into four parts:

1. **SETTING THE CRITERIA:** Initially, the CDE team established the criteria to be used later on during the labeling phase. Instead of relying solely on classic RGB images for cloud segmentation, the team used various combinations of bands provided by the Sentinel 1 & 2 satellites. For instance, in certain scenarios where the clouds might easily be mistaken for a snowy cliff or mountain top, the combination of the SWIR, B11 and B12 bands, from the Sentinel 2 satellite, can help distinguish between the clouds and the snow.

2. **HUMAN CALIBRATION:** Once the criteria and some examples were established, each member of the CDE team had to be trained to ensure consistency and accuracy in their individual annotations of different IPs. In other words, the annotators needed to be "calibrated" to produce results that were as high-quality, fair, and similar as possible to those of their peers.

3. **LABELING/ANNOTATION:** Each member of the CDE team was assigned a distinct set of IPs and then they each generated different masks for all the different IPs. It is important though, that each member must create the masks independently. While they may utilize assistance from the different CDAs and similar tools, there must be no collaboration within the members, because that would jeopardize the quality control phase.

4. **QUALITY CONTROL:** Despite human calibration, errors can still occur, which is why a Quality Check System (QCS) is implemented. Initially, an automatic check using a gradient boosting decision tree is performed. Subsequently, CDE team members assume the role of inspectors and review the work of their peers. Each mask undergoes evaluation by at least two

inspectors, with additional inspectors involved for more challenging masks to ensure comprehensive assessment. Finally, each IP is assigned a difficulty label ranging from 1 to 5. A difficulty level of 1 indicates high confidence from the CDE team in the resulting mask, while a level of 5 suggests lower confidence, yet still sufficient for inclusion in the dataset.

To assist with the annotation process and reduce overall annotation time, the CDE team utilized the IRIS tool from ESA[3].

3.3 Files and Directories

Once the annotation was completed, it was time to organize the results into the corresponding files. When navigating through the dataset, we differentiate between four distinct levels:

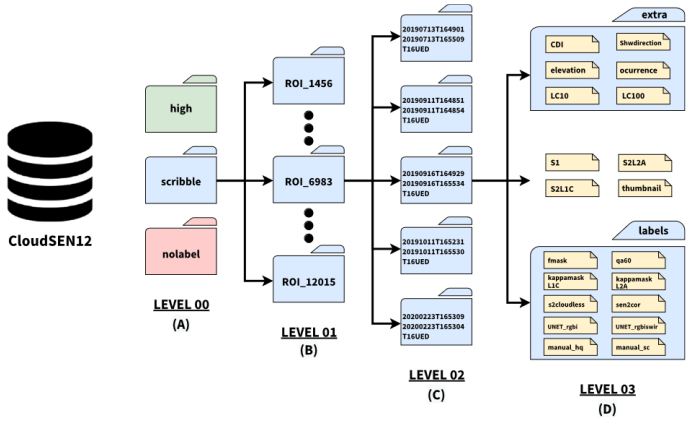


Figure 1: Folder structure of the CloudSEN12 dataset (image from[1])

- **Level zero (A): Annotation Quality of Each ROI.** There are three levels of annotation quality:
  - High-Quality: These masks index thin and thick clouds, as well as shadows and clear skies. These are the annotations we will be using.
  - Scribble: These ROIs only contain annotations regarding the presence or absence of clouds. They could be useful in a possible implementation of the project, where we focus solely on identifying clouds.
  - No-Anotation: These ROIs lack detailed masks. They only indicate whether there is a cloud in the image but do not specify its region.
- **Level One (B): Region of Interest (ROI).** After selecting the desired level of annotation quality, we can access the ROIs.
- **Level Two (C): Image Patches (IPs).** Within each ROI, there are five IPs to choose from.
- **Level Three (D): Detailed Information for Each IP.** Here, we find everything related to each IP. Specifically, we can find:
  - Sentinel images: without going further into any subfolders, we can find all the Sentinel images. Specifically:
    - S1: Sentinel 1 images
    - S2L2A: Sentinel 2 Level 2 A

- S2L1C: Sentinel 2 Level 1 C (the one we will be using)
- thumbnail

- labels: This folder contains all the different masks generated by the CDE team and various CDAs. The "manual_hq" masks, created by the CDE team, will be used as the ground truth.
- extras: This folder includes additional information from the Sentinel 1 and 2 satellites, such as elevation data or shadow direction. This folder will not be used for this project.

File/ Folder	Name	Scale	Wavelength	Description
S2L1C & S2L2A	B1	0.0001	443.9 nm (S2A)/442.3 nm (S2B)	Aerosols.
	B2	0.0001	496.6 nm (S2A)/492.1 nm (S2B)	Blue.
	B3	0.0001	560 nm (S2A)/559 nm (S2B)	Green.
	B4	0.0001	664.5 nm (S2A)/665 nm (S2B)	Red.
	B5	0.0001	703.9 nm (S2A)/703.8 nm (S2B)	Red Edge 1.
	B6	0.0001	740.2 nm (S2A)/739.1 nm (S2B)	Red Edge 2.
	B7	0.0001	782.5 nm (S2A)/779.7 nm (S2B)	Red Edge 3.
	B8	0.0001	835.1 nm (S2A)/833 nm (S2B)	NIR.
	B8A	0.0001	864.8 nm (S2A)/864 nm (S2B)	Red Edge 4.
	B9	0.0001	945 nm (S2A)/943.2 nm (S2B)	Water vapor.
	B11	0.0001	1613.7 nm (S2A)/1610.4 nm (S2B)	SWIR 1.
	B12	0.0001	2202.4 nm (S2A)/2185.7 nm (S2B)	SWIR 2.
S2L1C	B10	0.0001	1373.5 nm (S2A)/1376.9 nm (S2B)	Cirrus.
S2L2A	AOT	0.001	—	Aerosol Optical Thickness.
	WVP	0.001	—	Water Vapor Pressure.
	TCI_R	1	—	True Color Image, Red.
	TCI_G	1	—	True Color Image, Green.
	TCI_B	1	—	True Color Image, Blue.
S1	VV	1	5.405 GHz	Dual-band cross-polarization, vertical transmit/ horizontal receive.
	VH	1	5.405 GHz	Single co-polarization, vertical transmit/vertical receive.
	angle	1	—	Incidence angle generated by interpolating the 'incidenceAngle' property.
extra/	CDI	0.0001	—	Cloud Displacement Index[41].
	Shwdirection	0.01	—	Azimuth. Values range from 0°–360°
	elevation	1	—	Elevation in meters. Obtained from MERIT Hydro datasets[38].
	ocurrence	1	—	JRC Global Surface Water[39]. The frequency with which water was present.
	LC100	1	—	Copernicus land cover product. CGLS-LC100 Collection 3[40].
	LC10	1	—	ESA WorldCover 10 m v100 product.
labels/	fmask[21]	1	—	Fmask4.0 cloud masking.
	QA60	1	—	SEN2 Level-1C cloud mask.
	s2cloudless[22]	1	—	sen2cloudless results.
	sen2cor	1	—	Scene Classification band. Obtained from SEN2 level 2A.
	CD-FCNN-RGBI	1	—	López-Puigdollers *et al.*[23] results based on RGBI bands.
	CD-FCNN-RGBISWIR	1	—	López-Puigdollers *et al.*[23] results based on RGBISWIR bands.
	kappamask_L1C	1	—	KappaMask[24] results using SEN2 level L1C as input.
	kappamask_L2A	1	—	KappaMask[24] results using SEN2 level L2A as input.
	manual_hq	1		High-quality pixel-wise manual annotation.
	manual_sc	1		Scribble manual annotation.

Figure 2: List of assets available for each image patch. (image from[4])

3.4 Downloading the Dataset

If you want to make use of the dataset, there are a couple of options available:

3.4.1 Downloading the Premade Memmaps

The CloudSen12 team has created various memmaps to facilitate our tasks, which can be found here [4]. The memmaps are categorized into training, validation, and testing groups, with 8,490, 535, and 975 IPs in each group, respectively. Each group contains as many memmaps as there are bands in the images. For example, if you want to use the S2L1C images, which have 13 bands, there will be a total of 39 memmaps—13 for each group (training, validation, and testing).

Using the data in this manner allows us to compare our results more accurately with those developed by the CloudSen12 team, as we use the same training, validation, and testing sets. However, this method has its drawbacks; for instance, we cannot identify individual IPs, making it quite challenging, or nearly impossible, to extract a specific subset of memmaps.

3.4.2 Downloading batches of ROIs directly.

Downloading the data this way can be more versatile, as it allows us to make queries based on different metadata associated with each IP and then create subsets of IPs. This flexibility enables the creation of different subsets, making it more useful for targeted analysis.

Metadata name	Description
annotator_name	The labeler's name.
roi_id	The region of interest ID.
s2_id_gee	Sentinel-2 GEE ID.
s2_id	Sentinel-2 product ID.
s2_date	Sentinel-2 acquisition date in ISO format.
s2_sen2cor_version	Sen2Cor configuration baseline used at the time of the product generation.
s2_fmask_version	Fmask version.
s2_s2cloudless_version	s2cloudless version.
s2_reflectance_conversion_correction	Earth-Sun distance correction factor.
s2_aot_retrieval_accuracy	Accuracy of aerosol optical thickness model.
s2_water_vapour_retrieval_accuracy	Declared accuracy of the Water Vapor model.
s2_view_off_nadir	The angle from the SEN2 sensor between nadir (straight down) and the scene center.
s2_view_sun_azimuth	SEN2 sun azimuth angle.
s2_view_sun_elevation	SEN2 sun elevation angle.
s1_id	SEN1 product ID.
s1_date	SEN1 acquisition date in ISO format.
s1_grd_post_processing_software_name	Name of the software to pre-processing SEN1.
s1_grd_post_processing_software_version	SEN1 software pre-processing version.
s1_slc_processing_facility_name	Name of the facility where the processing step was performed.
s1_slc_processing_software_version	Software version identification.
s1_radar_coverage	percentage of valid SEN1 pixels contained in this IP.
land_cover	Predominant land use.
label_type	Manual labeling type (i.e., scribble, high-quality or no-annotation).
cloud_coverage	Cloud coverage estimated using photo-interpretation. (see Method: Image patches selection section).
test	Whether the IP is part of training (train) or testing (test) dataset.
difficulty	Labeler's confidence (from 1 to 5) of the manual annotation.Where one indicates near-perfect and five denotes potentially significant mistakes.
proj:epsg	EPSG code.
proj:geometry	Footprint of this IP.
proj:shape	Number of pixels for the default IP.
proj:centroid	Centroid coordinates of the IP in latitude and longitude.
proj:transform	The affine transformation coefficients.

Figure 3: Metadata associated to each image patch. (image from[1])

This approach does have its problems: first, it involves downloading a substantial amount of data, some of which migh be redundant for our project purpose. Additionally, if we wanted to compare our model with others, this method wouldn't be reliable because we wouldn't have the same training or testing images. There is also the issue that the data is raw: the images are in the shape of $509x509xbands$, which can pose a problem for our model and would therefore require padding. Moreover, we would need to create the memmaps ourselves. Fortunately, the code for adapting these images has already been created and can be found in the repository[6].

All of this data can be downloaded here [5].

3.4.3 Making our oun memmaps

This third option is more of a continuation of the second one. Instead of making queries, reading, padding, one-hot encoding, and then creating the memory maps every time we want to train or test the models, we can perform this process just once, store the generated memmaps, and reuse them in future attempts.

This approach has the same issues as downloading batches of ROIs directly: the images are not the same as those from the CloudSen12 team, rendering them unusable for comparison, and it involves

downloading a lot of redundant data. However, it has the benefit that when doing extensive training or testing without changing the data, this approach is faster and less time-consuming.

In the repository of this project, you can find all of the scripts and code that facilitate these approaches.

4 Model

This section will be dedicated to the model. We will discuss the following topics:

- The **model architecture** implemented in the project so far.
- The metrics used for **comparing different models**.
- A brief description of **our trained models**.

4.1 Model Architecture (U-Net)

The developed model is based on a U-Net architecture. A U-Net is a type of convolutional neural network (CNN) architecture designed primarily for image segmentation. It was introduced by Olaf Ronneberger, Philipp Fischer, and Thomas Brox in their 2015 paper "U-Net: Convolutional Networks for Biomedical Image Segmentation"[7]. The architecture is named U-Net due to its U-shaped structure, which is formed by a contracting path (encoder) and an expansive path (decoder).

The reason for this choice is that U-Net has proven to be a useful architecture in many scenarios. In fact, one of the example models provided by the CloudSen12 team is a U-Net very similar to ours. This convinced us to choose the U-Net architecture for the model, at least initially.

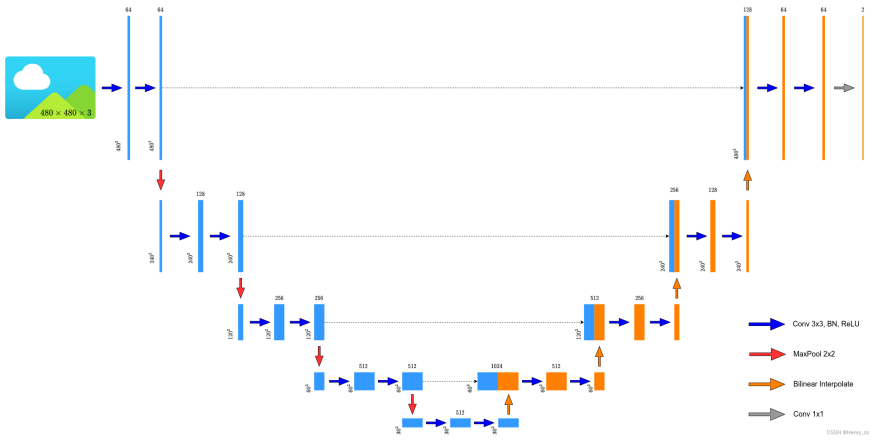


Figure 4: Example of a U-Net architecture, note that this is schematic does not represent our model.

4.1.1 Input Layer

Without any mystery, the input layer will take an image divided by bands so that it can be used later on. The shape of the input image must be $512x512xbands$, where "*bands*" represents the number of bands the image has. The number of bands must be determined during the creation of the model.

Algorithm 1 Tensorflow python code for the Input Layer (full code at [6])

```
# Input
input_layer = Input((512,512,num_bands))
```

4.1.2 Contracting path (Encoder)

This path consists of a series of convolutional layers followed by max-pooling operations. Each step in this path halves the dimensions while increasing the number of feature channels, thereby capturing context and features at different levels of abstraction.

In our case, the encoder will be six steps deep, with each step consisting of two consecutive 2D convolutional layers (*Conv2D*), each with a $3x3$ kernel, followed by a 2D max-pooling layer (*MaxPooling2D*).

Algorithm 2 Tensorflow python code for the Encoder (full code at [6])

```
encoders_conv = []
encoders_MaxP = []

# Layer 1
encoders_conv.append(Conv2D(64, 3, activation='relu', padding='same')(input_layer))
encoders_conv.append(Conv2D(64, 3, activation='relu', padding='same')(encoders_conv[-1]))
encoders_MaxP.append(MaxPooling2D(pool_size=(2,2))(encoders_conv[-1]))

# Layer 2
encoders_conv.append(Conv2D(128, 3, activation='relu', padding='same')(encoders_MaxP[-1]))
encoders_conv.append(Conv2D(128, 3, activation='relu', padding='same')(encoders_conv[-1]))
encoders_MaxP.append(MaxPooling2D(pool_size=(2,2))(encoders_conv[-1]))

# Layer 3
encoders_conv.append(Conv2D(256, 3, activation='relu', padding='same')(encoders_MaxP[-1]))
encoders_conv.append(Conv2D(256, 3, activation='relu', padding='same')(encoders_conv[-1]))
encoders_MaxP.append(MaxPooling2D(pool_size=(2,2))(encoders_conv[-1]))

# Layer 4
encoders_conv.append(Conv2D(512, 3, activation='relu', padding='same')(encoders_MaxP[-1]))
encoders_conv.append(Conv2D(512, 3, activation='relu', padding='same')(encoders_conv[-1]))
encoders_MaxP.append(MaxPooling2D(pool_size=(2,2))(encoders_conv[-1]))

# Layer 5
encoders_conv.append(Conv2D(1024, 3, activation='relu', padding='same')(encoders_MaxP[-1]))
encoders_conv.append(Conv2D(1024, 3, activation='relu', padding='same')(encoders_conv[-1]))
encoders_MaxP.append(MaxPooling2D(pool_size=(2,2))(encoders_conv[-1]))

# Layer 6
encoders_conv.append(Conv2D(1024, 3, activation='relu', padding='same')(encoders_MaxP[-1]))
encoders_conv.append(Conv2D(1024, 3, activation='relu', padding='same')(encoders_conv[-1]))
encoders_MaxP.append(MaxPooling2D(pool_size=(2,2))(encoders_conv[-1]))
```

4.1.3 Bottleneck

At the bottom of the U, there is a bottleneck that connects the contracting path to the expansive path. Our bottelneck is not very complex, is made out of just two 2D convolution layers, which in turn are made out of 1024 kernels of shape $3x3$.

Algorithm 3 Tensorflow python code for the Bottleneck (full code at [6])

```
bn_layers = []

bn_layers.append(Conv2D(1024, 3, activation='relu', padding='same')(encoders_MaxP[-1]))
bn_layers.append(Conv2D(1024, 3, activation='relu', padding='same')(bn_layers[-1]))
```

4.1.4 Expanding path (Decoder)

The decoder is very similar to the encoder but in reverse. It uses up-convolutions or transposed convolutions (Conv2DTranspose) to increase the dimensions. Each step in this path includes a concatenation layer with the corresponding feature map from the contracting path. These so-called skip connections help retain spatial information that might be lost during down-sampling in the contracting path. One can also add additional convolutional layers before or after the concatenate layer.

In our case, we have decided to add a convolutional layer after the concatenation. The kernels will be the same size for both Conv2DTranspose and the convolutional layers, but the number of kernels will differ from the encoder, resulting in fewer kernels in the decoder at each corresponding step.

Algorithm 4 Tensorflow python code for the Decoder (full code at [6])

```
decoders_conv_t = []
decoders_conc = []
decoders_conv = []


# Layer 6'
decoders_conv_t.append(Conv2DTranspose(1024, (2, 2), strides=(2,2),padding='same')(bn_layers[-1]))
decoders_conc.append(concatenate([decoders_conv_t[-1], encoders_conv[-1]], axis=3))
decoders_conv.append(Conv2D(1024, 3, activation='relu', padding='same')(decoders_conc[-1]))

# Layer 5'
decoders_conv_t.append(Conv2DTranspose(512, (2, 2), strides=(2,2), padding='same')(decoders_conv[-1]))
decoders_conc.append(concatenate([decoders_conv_t[-1], encoders_conv[-3]], axis=3))
decoders_conv.append(Conv2D(512, 3, activation='relu', padding='same')(decoders_conc[-1]))

# Layer 4'
decoders_conv_t.append(Conv2DTranspose(256, (2, 2), strides=(2,2), padding='same')(decoders_conv[-1]))
decoders_conc.append(concatenate([decoders_conv_t[-1], encoders_conv[-5]], axis=3))
decoders_conv.append(Conv2D(256, 3, activation='relu', padding='same')(decoders_conc[-1]))

# Layer 3'
decoders_conv_t.append(Conv2DTranspose(128, (2, 2), strides=(2,2), padding='same')(decoders_conv[-1]))
decoders_conc.append(concatenate([decoders_conv_t[-1], encoders_conv[-7]], axis=3))
decoders_conv.append(Conv2D(128, 3, activation='relu', padding='same')(decoders_conc[-1]))

# Layer 2'
decoders_conv_t.append(Conv2DTranspose(64, (2, 2), strides=(2,2), padding='same')(decoders_conv[-1]))
decoders_conc.append(concatenate([decoders_conv_t[-1], encoders_conv[-9]], axis=3))
decoders_conv.append(Conv2D(64, 3, activation='relu', padding='same')(decoders_conc[-1]))

# Layer 1'
decoders_conv_t.append(Conv2DTranspose(32, (2, 2), strides=(2,2), padding='same')(decoders_conv[-1]))
decoders_conc.append(concatenate([decoders_conv_t[-1], encoders_conv[-11]], axis=3))
decoders_conv.append(Conv2D(32, 3, activation='relu', padding='same')(decoders_conc[-1]))
```

4.1.5 Output Layer

The main purpose of this layer is to reshape the data into its final shape. This final shape will be $512x512xclasses$, where "classes" represents the number of classes to identify. We usually deal with four classes, so one must be careful when dealing with a different number of classes. It's not that the model isn't prepared for other numbers of classes—it is—but the images are automatically one-hot-encoded in various parts of the code. This means they frequently change their shape to $512x512x4$ across different scripts. This could cause problems when testing or training the code, so I advise caution.

Algorithm 5 Tensorflow python code for the Output Layer (full code at [6])

```
# Output
output_layer = Conv2D(num_classes, 1, activation='softmax')(decoders_conv[-1])
```

4.2 Comparing Different Models

To analyze the quality of a trained model quantitatively, we will use a couple of metrics proposed by the CloudSen12 team:

- **Producer's Accuracy (PA):** High PA values show that cloud pixels have been effectively masked out. We like high values.

$$PA = \frac{TP}{TP + FN}$$

where: $TP = True\ Positives$, $FN = False\ Negatives$.

- **User's Accuracy (UA):** High UA values indicate that the model is cautious in excluding non-cloud pixels. We like high values.

$$UA = \frac{TP}{TP + FP}$$

where: $TP = True\ Positives$, $FP = False\ Positives$.

We will differentiate three subclasses of the PA and UA metrics: low (< 0.1), mid ($0.1 < mid < 0.9$), and $high$ (> 0.9). For each IP, we will calculate the metrics and then assign them to a specific subclass based on their values. For example, if for a certain IP we have $PA = 0.95$ and $UA = 0.25$, we will increase the count of PA_{high} and UA_{mid} by one. Once all the IPs are processed, we will calculate the proportions for each subclass.

- **Balanced Overall Accuracy (BOA):** High BOA values are related to a good balance of false positives and false negatives.

$$BOA = 0.5 \cdot (PA + \frac{TN}{TN + FP})$$

where: $TN = True\ Negatives$, $FP = False\ Positives$.

Once we've determined the metrics we will use, we will set up different scenarios to analyze the behavior of our model under various circumstances. These scenarios are:

1. **Scenario 1:** Each pixel of an image will be divided into one of the following sub-classes:
   - **Cloud** (Positives): Comprised of all thick and thin cloud pixels.
   - **No-cloud** (Negatives): Comprised of all non-cloud and shadow pixels.
2. **Scenario 2:** Each pixel of an image will be divided into one of the following sub-classes:
   - **Cloud or shadow** (Positives): Comprised of all thick cloud, thin cloud, and shadow pixels.
   - **No-cloud** (Negatives): Comprised of all non-cloud pixels.

Other metrics can be used for other purposes, such as differentiating between thick and thin clouds. However, these two scenarios are the ones proposed by the CloudSen12 team. This allows for easier comparison since we have all the results from the different CDAs.

4.3 Our Trained Models

In total, we have trained three distinct models, which can be considered evolutions from their predecessors.

1. **Model 1:** This is the first and simplest model, consisting of a U-Net with a depth of 3 instead of 6, although the architecture is the same as the one explained earlier. Additionally, the training images have been selected via a query and are not the same as those used by the CloudSen12 team. Consequently, the number of training images is significantly lower compared to using the predefined memmaps.
   For the training parameters, we decided to use Categorical Cross-Entropy as the loss function and trained the model for 300 epochs with a batch size of 4.

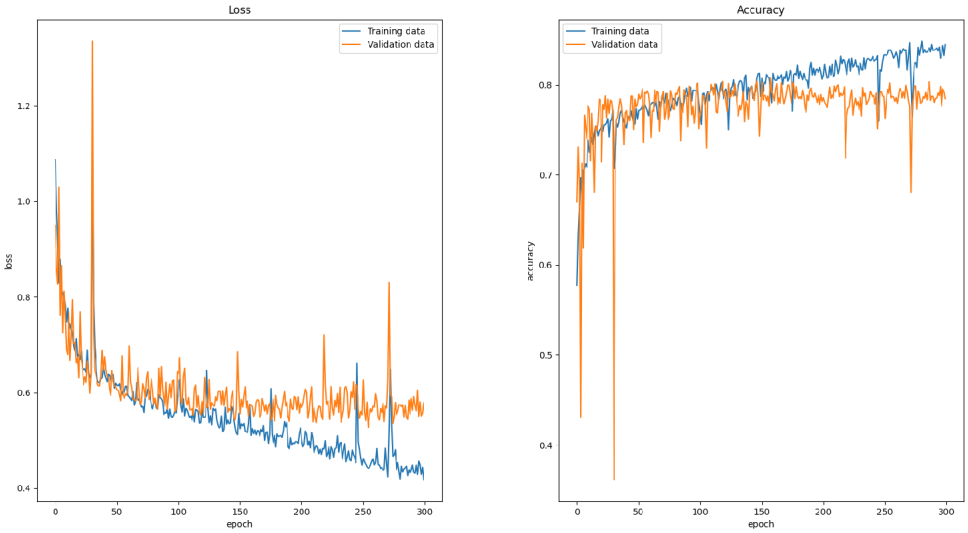


Figure 5: Model 1's loss function and accuracy history.

   We can clearly see that we've reached the overfitting point around the 100th epoch, with an approximate accuracy of 79%.

2. **Model 2.0:** This model has a depth of 6 layers, similar to our initial description, but the training data and training parameters remain the same as in Model 1.

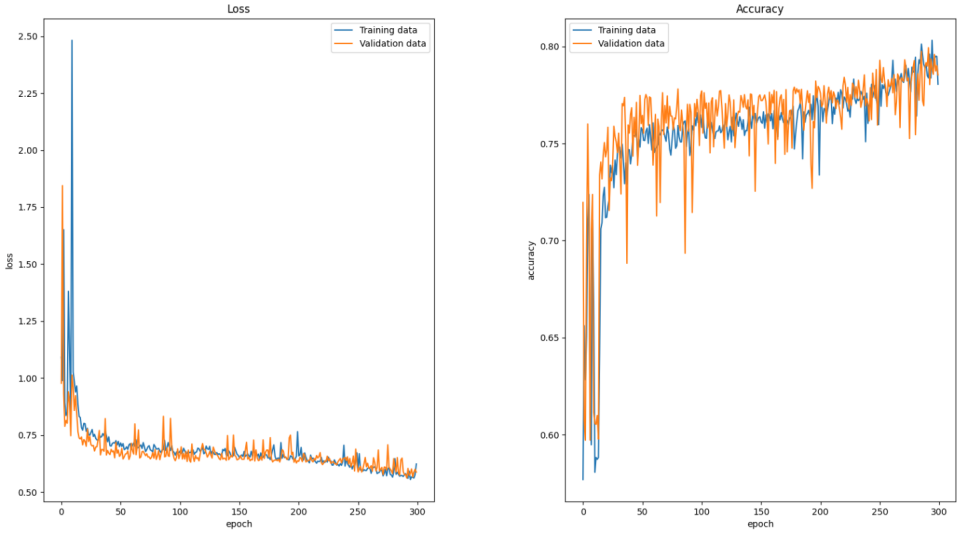


Figure 6: Model 2.0's loss function and accuracy history.

Being this a much bigger model than the previous one, it is expected that it took more epochs to train. In fact, the overfitting point has not yet been reached and doesn't seem imminent. This indicates that this version of the model would benefit from a longer training time.

3. **Model 2.1:** This model cannot be interpreted as an update over Model 2.0. The only difference is that this time we have used the memmaps as training data instead of the query-selected images, and the batch size has been increased to 16 images instead of 4. For this reason, when comparing our models with the CloudSen12 ones, we will place more trust in the results from this model.

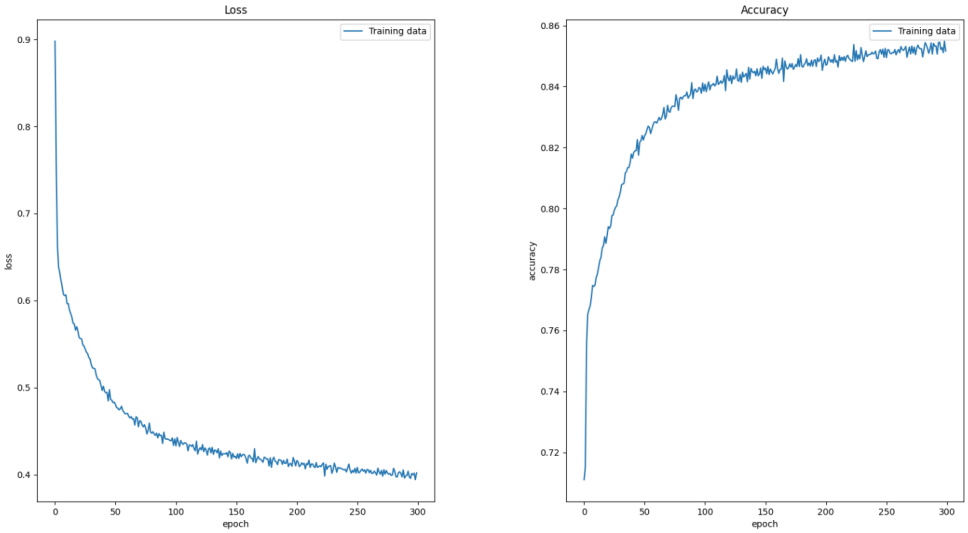


Figure 7: Model 2.1's loss function and accuracy history.

Even though the validation data cannot be seen due to a problem with the code, we can estimate this to be a better version than Model 2.0, at least up to the 300th epoch. The main reason is

that the maximum accuracy of the outdated model was around 80%, whereas the new model's accuracy surpassed 80% at an epoch where we wouldn't expect the overfitting point to occur.

5 Results

Having explained all the trained models, we will now present the results obtained so far. We will examine the results from three different perspectives:

- Quantitative Analysis: In this approach, we will apply the previously explained metrics to all the models and compare them with the CloudSen12 models.
- Empirical Analysis: In this approach, we will examine various generated masks to identify possible problems and areas for improvement.
- Confusion Matrixes: In this aproach, we will analize the confusion matrices generated by our models, and try then to identify areas of improvement.

After that, we will discuss possible changes or upgrades to the model or the training process to achieve better results.

5.1 Quantitative Analysis

For this analysis, we will use the scenarios and metrics explained in the previous sections. The results will be displayed in tables, from which we will extract conclusions. It is important to note that the results of Model 1 and Model 2.0 have not been taken into account, as they were trained and tested with different data from the other models, making any comparison unfair.

5.1.1 Scenario 1

In this scenario, each pixel of an image will be classified as either cloud (thick and thin clouds) or non-cloud (non-clouds and shadows). The former will be treated as positive pixels, and the latter as negative pixels.

Model	BOA	$PA_{low}\%$	$PA_{mid}\%$	$PA_{high}\%$	$UA_{low}\%$	$UA_{mid}\%$	$UA_{high}\%$
Human Level	0.99	1.03	14.03	84.94	0.13	4.39	95.48
UNetMobV2	0.92	0.77	30.63	68.6	0.26	25.03	74.71
KappaMask L2A	0.77	2.83	31.92	65.25	1.56	63.04	35.41
KappaMask L1C	0.82	4.89	45.3	49.81	0.65	38.38	60.97
Fmask	0.84	5.92	40.54	53.54	0.26	52.65	47.09
s2cloudless	0.79	7.08	52.38	40.54	0.65	31.5	67.84
Sen2Cor	0.71	13.13	64.86	22.01	1.58	20.05	78.36
QA60	0.58	24.84	49.94	25.23	1.39	37.62	60.99
CD-FCNN-RGBI	0.72	17.50	74.00	8.49	1.62	12.58	85.79
CD-FCNN-RGBISWIR	0.72	18.40	71.43	10.17	0.82	9.43	89.75
Model 2.1	0.47	41.33	52.00	6.67	29.45	62.97	7.59

Table 1: First scenario results (cloud, no-cloud)

From these results, we can determine that our models are not performing well, as their Balanced Overall Accuracy (BOA) is still very low compared to the CloudSen12 models.

5.1.2 Scenario 2

In this scenario, each pixel of an image will be classified as either cloud-shadow (thick cloud, thin cloud, and shadow) or non-cloud. The former will be treated as positive pixels, and the latter as negative.

Model	BOA	$PA_{low}\%$	$PA_{mid}\%$	$PA_{high}\%$	$UA_{low}\%$	$UA_{mid}\%$	$UA_{high}\%$
Human level	0.99	1.03	14.8	84.17	0.13	2.33	97.55
UNetMobV2	0.91	0.77	28.57	70.66	0.00	17.14	82.86
KappaMask L2A	0.75	2.96	39.38	57.66	1.29	44.32	54.39
KappaMask L1C	0.81	3.99	47.62	48.39	0.65	32.64	66.71
Fmask	0.81	4.89	45.43	49.68	0.26	44.34	55.39
Sen2Cor	0.67	13.77	69.63	16.6	1.05	18.58	80.37
Model 2.1	0.47	11.69	62.46	25.85	10.26	62.15	27.59

Table 2: Second Scenario Results

As with the previous scenario, our results are not that great, as they still lag significantly behind the CloudSen12 models.

5.1.3 Confusion Matrices

Aside from the different analyzed scenarios, we have also deemed it appropriate to examine the confusion matrix from our model. The aim of this analysis is to evaluate each model's individual performance and identify weak spots and/or areas for improvement.

	no-cloud	thick-cloud	thin-cloud	shadow
no-cloud	79'803'924	40'900'771	12'963'851	11'500'032
thick-cloud	34'528'285	20'031'673	29'996'683	5'007'175
2459979thin-cloud	10'872'741	7'297'087	1'774'069	2'459'979
shadows	1'305'086	7'356'754	2'411'378	2'167'912

Table 3: Confusion matrix from Model 2.1. All pixels representation. Rows represent the ground truth, and the columns represent the model's predictions.

	no-cloud	thick-cloud	thin-cloud	shadow
no-cloud	57.57	54.11	63.99	54.41
thick-cloud	24.91	26.50	14.91	23.69
thin-cloud	7.84	9.65	9.30	11.63
shadows	9.67	9.73	11.90	10.25

Table 4: Confusion matrix from Model 2.1. Percentage representation. Rows represent the ground truth, and the columns represent the model's predictions.

As we can see, the results don't appear to be that good. Let's isolate a couple of problems:

- The model has a tendency to predict a pixel as no-cloud. This is not surprising as this is by far the largest class. A lack of training might explain this symptom.
- A lot of thin-clouds don't get classified. This might be because this class is not very recurrent, and when looking at the big picture, if 80% of an image is no-cloud and only 20% is cloud, then

if the model predicts all as no-cloud, that's an 80% accuracy. Additionally, thin-clouds tend to be quite small and sparse, making them harder to predict. A change in the loss function might help with this problem.

- There are a lot of no-clouds that get classified as thick-clouds. This might be due to poor definition of the borders between the clouds and the no-clouds in the predictions. A combination of a lack of training and a loss function that doesn't emphasize border definition might explain this problem.
- The shadows are not well defined. They might face the same problem as thin-clouds, although they should be easier to identify.

5.2 Empirical Analysis

Now we will look at different IP images (RGB), masks (manual_hq), and predictions of Model 2.1 (predictions) to see if we can spot different problems that the numbers aren't revealing. The color palette used in the images is as follows:

- **Purple**: No-cloud
- **Light green**: Thin-cloud
- **Blue**: Thick-cloud
- **Yellow**: Shadow

5.2.1 Wrong Behaviors

The trained model has problems with the following scenarios:

- **Mistaking thin-clouds as shadows**: Unfortunately, this mistake happens more often than not. The problem might be a spectral one, as thin-clouds might project shadows below them, which can confuse the model. Additionally, both shadows and thin-clouds tend to appear next to thick clouds, making it easier to make mistakes. More training might prove useful.
- **Mistaking fields and crops for thin-clouds**: The thin-cloud layer is proving to be the most difficult one, as it can also be mistaken for vegetation, especially crops. This might be due to fertilizers or the type of crop. Fortunately, this doesn't happen frequently, and a bit more training could help mitigate it.
- **Shadows from non-cloud sources**: Clouds are not the only ones to project shadows. Sometimes, abrupt changes in elevation can cause shadows which, spectrally, look the same as cloud shadows, leading to misclassification. This should not pose a significant problem, as large shadow projectors are necessary to be detectable on satellite images, but for mountainous terrain, this could be an issue.

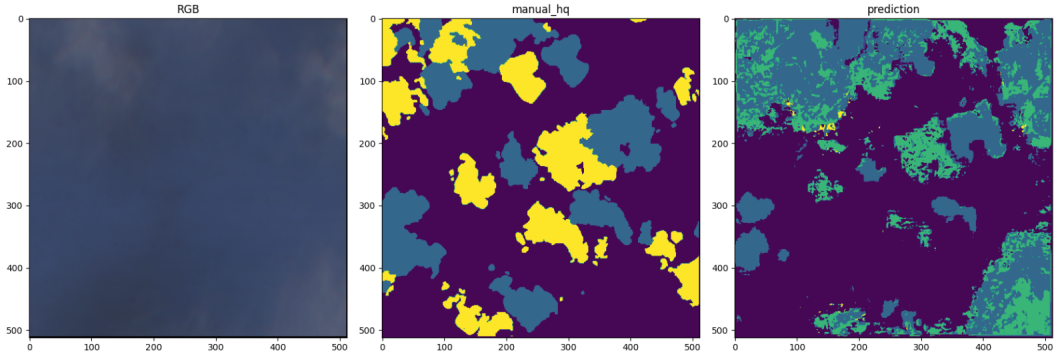


Figure 8: Prediction that mistakes thin-cloud pixels for shadow ones.

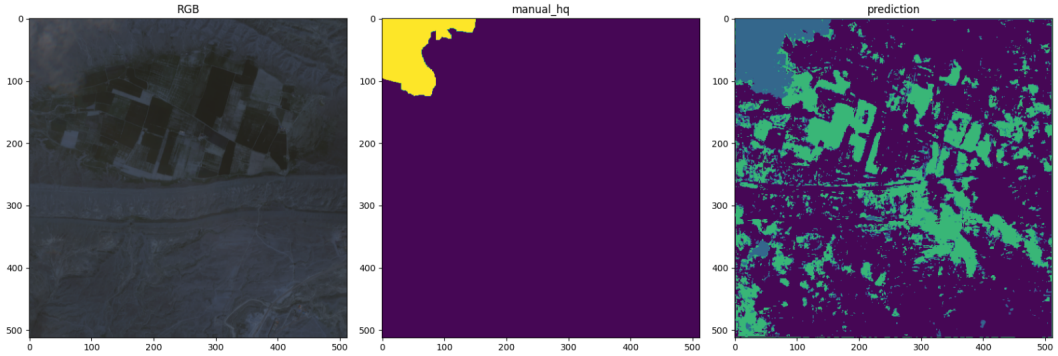


Figure 9: Prediction that mistakes thin-clouds for vegetation

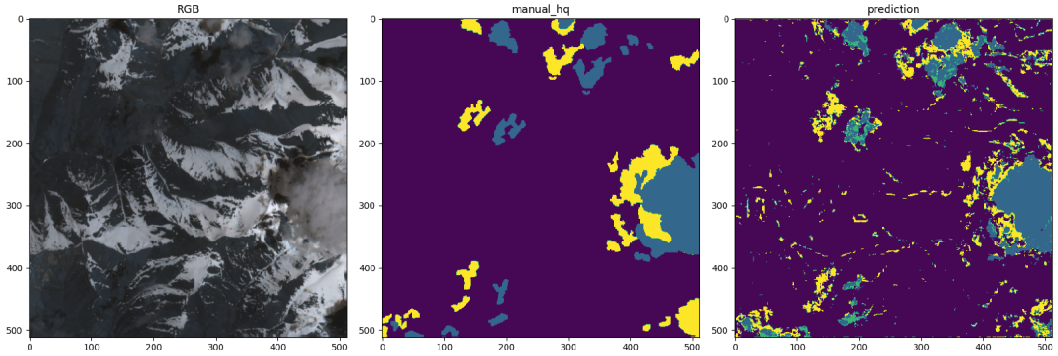


Figure 10: Prediction that mistakes cloud shadows for shadows projected by changes in terrain elevation.

5.2.2 Good Behaviors

To be fair to the model, there are a couple of things it gets right:

- **Snow does not pose a problem:** The similarity between snow and clouds was a big concern initially, but the model has been able to segment them quite well.

- **Images composed mainly of thick-clouds are reasonably well segmented**: Although the borders of the clouds are seldom well defined, the model segments thick-cloud areas quite accurately.

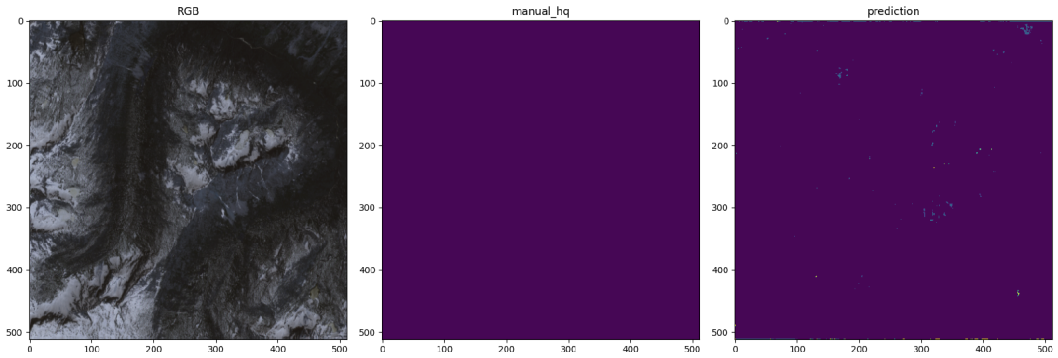


Figure 11: Prediction that correctly dismisses snow.

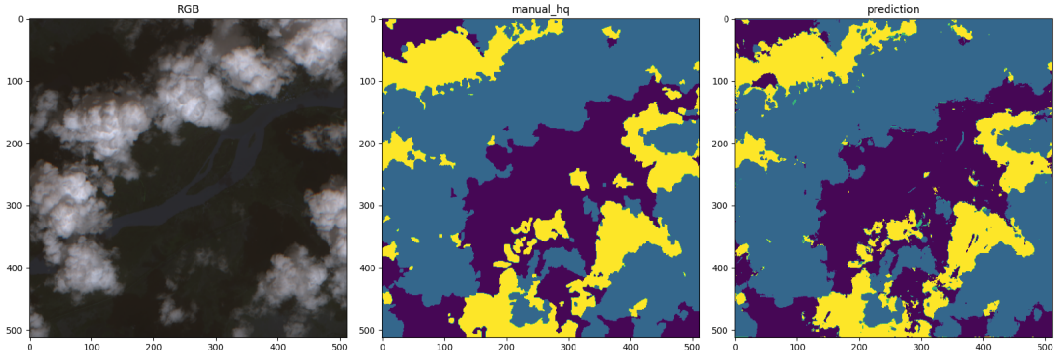


Figure 12: Prediction that correctly segments an image with thick-clouds.

Another aspect to keep in mind is that sometimes the masks are not entirely accurate and can occasionally be wrong altogether. However, this happens very rarely.

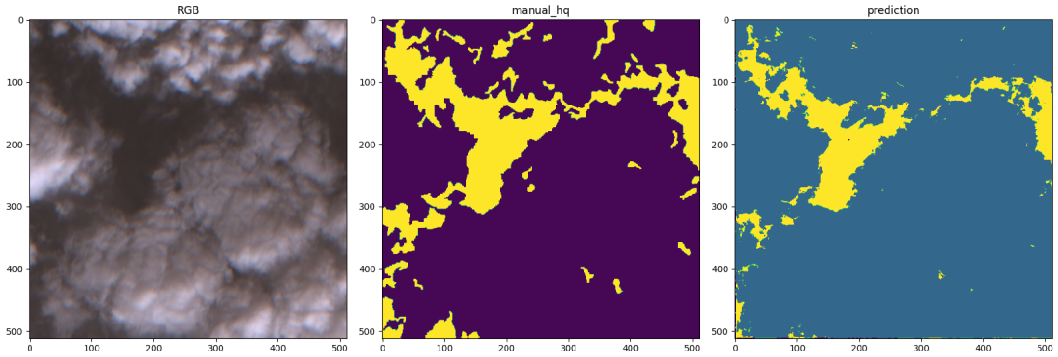


Figure 13: Wrongly defined mask.

5.3 Upgrades and Changes

As we've seen, the results from our models are not that great, so improvements are in order. In this subsection, we will inspect some changes we could make to achieve better results and create better models overall.

Before we continue, we would like to state that even though some of these changes might seem trivial, they haven't been implemented as of the writing of this report. The main reasons are a combination of a lack of time and experience on the intern's side.

5.3.1 Training Changes

One improvement that could make a difference when training the model is changing the loss function. So far, we've only used categorical cross-entropy, which is good, but it might be more useful to use one of the following:

- **Intersection over Union (IoU)**: We've observed that the borders between classes in our predictions are not very well defined. Even in the best cases, there are many small errors, and the current loss function does not adequately account for these borders. Using IoU could improve the predictions in this aspect.
- **The metrics proposed by the CloudSen12 team (PA, UA, or BOA)**: Although this might not be the most conventional approach, it would certainly make the model compare better to the CloudSen12 models.
- **More training time**: We've trained all the models for 300 epochs, equivalent to about 2.5 days. This might not be enough training time, so we should try to optimize the number of epochs. Unfortunately, the code for the validation data during training with the CloudSen12 memmaps is not complete and would need some fixing.

5.3.2 Model Changes

As stated before, we do not think the model requires significant improvement, and we might be able to identify more issues once the training changes have been implemented. That said, during the development of the project, we've come across this paper[8], which addresses the same problem as ours but with a more complex approach regarding the model. Additionally, we can try to improve the current model through small tweaks to the parameters of the different layers.

6 Repository

To conclude this report, I would like to introduce the repository to ensure there are no confusions. Although the repository itself is not difficult to navigate, a brief overview will help clarify its structure and contents.

6.1 File structure

It's important to follow the file/folder structure of the repository strictly, as some functions rely on specific paths. Note that due to file size constraints, some files or folders might not appear in our GitHub repository

Directories		Contents
* _memmaps/		Contains all of the memmaps, except for the CloudSen12 ones.
* _saved_history/		Contains each trained model's history, saved as .pkl files.
* _saved_models/		Contains all the models, saved as .keras files.
Code/	.	This directory stores all of the project's code.
	Functions/	Contains functions that are used throughout the different scripts.
	Modules/	Contains the Model and the Dataloader.
	Scripts/	Contains code that can be directly executed.
Datasets/	.	This directory stores all of the project's dataset information.
	* _memmaps/	Contains CloudSen12 memmaps.
	* CloudSen12_Dataset/	Contains the CloudSen12 dataset, indexed by ROIs.
	metadata/	Contains metadata files created by the make_metadata_files.py script.
	* CloudSen12_Dataset.zip	Zip file of the CloudSen12_Dataset/ directory.
	Dataset_download_links.txt	Contains all the download links for the CloudSen12 dataset (by ROIs).

Table 5: File/folder structure. Directories marked with a * can not be found on the GitHub repository.

6.2 Code

This subsection addresses different scenarios you might encounter when working with the code.

6.2.1 Executable code

If you want to train a new model, test an old one, or one-hot-encode some masks, there should be a script in the *Code/Scripts/* directory that does the job.

If there is a script that does the required job, the next step is to alter the parameters to fit the user's needs. These parameters can be found at the top of the script, allowing for easy execution without having to understand all of the ins and outs of the code.

If there is a need to create a new script to accommodate a new procedure, please do so in the *Code/Scripts/* directory and document it with comments on GitHub.

6.2.2 Changes to the model

At some point, we might want to change the model. While there is no dedicated script for this, there is a function responsible for creating new models. This function can be found in the *Code/Modules/-model.py* file. Changes can be made to this function to create new models.

Once you have modified the function, it is recommended to run the training_script.py script with the MODEL_FROM_0 flag activated. This will create the new model, train it, and then store it.

6.2.3 Loss Functions and Metrics

So far in the project, we haven't dealt extensively with different loss functions or metrics. Therefore, for the time being, you can find all of them in the *Code/Modules/model.py* file. There is room to implement this differently if one wishes.

6.2.4 Dataset and Dataloader

The *Code/Modules/dataloader.py* file might be the most confusing of all, as we have adopted the OOP paradigm and encapsulated all functions within a class. Generally, you should not need to directly interact with these functions. If problems related to the dataset arise, we recommend first checking the file or folder structure before dealing with the functions themselves.

There is an exception concerning the validation data from the CloudSen12 dataset that requires fixing.

6.3 Errors

Eaven though we have done everything at our disposal to minimize errors and conflicts inside the diferent files, there can still be some. That said we have identified a couple problems that need to be fixed.

Validation Data for CloudSen12 Memmaps Not Working
When training a model with the CloudSen12 pre-made memmaps, we use TensorFlow's tf.data.Dataset API rather than using the memmaps directly. This approach is necessary due to file size constraints that arise when dealing with large memmaps files.

While the training data processes successfully using tf.data.Dataset, the validation data fails to run properly. The issue may lie in how the validation data is fed into the fit function of the model.

Unmatching Masks for Testing Data:
There appears to be an issue with how we retrieve the testing data from the CloudSen12 memmaps. This problem becomes evident during qualitative and potentially quantitative testing, where some, but not all, images do not align with their corresponding masks.

This issue was detected toward the end of the student's internship, so it has not been fully addressed and may be more problematic than initially apparent.

References

[1] Aybar, C., Ysuhuaylas, L., Loja, J. et al. CloudSEN12, a global dataset for semantic understanding of cloud and cloud shadow in Sentinel-2. Sci Data 9, 782 (2022). https://doi.org/10.1038/s41597-022-01878-2

[2] Sentinels 1 and 2 band combinations, https://gisgeography.com/sentinel-2-bands-combinations/

[3] Mrziglod, J. IRIS - Intelligence foR Image Segmentation (2019).

[4] Hugging Face, download links for the CloudSen12 dataset high-quality memmaps: https://huggingface.co/datasets/csaybar/CloudSEN12-high/tree/main

[5] ScienceDataBank, download links for the CloudSen12 dataset: https://www.scidb.cn/en/detail?dataSetId=f72d622ff4ea4fa18070456a98222b1a

[6] Git-Hub repository URL: https://github.com/cloudsegmentation/jmoran

[7] Ronneberger, O., Fischer, P., Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. In: Navab, N., Hornegger, J., Wells, W., Frangi, A. (eds) Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015. MICCAI 2015. Lecture Notes in Computer Science(), vol 9351. Springer, Cham.

[8] Hu, K.; Zhang, D.; Xia, M. CDUNet: Cloud Detection UNet for Remote Sensing Imagery. Remote Sens. 2021, 13, 4533. https://doi.org/10.3390/rs13224533